

Fedimint Confidential eCash

Threshold-issued, value-hiding, unlinkable notes from Pointcheval–Sanders signatures

Gazelle

July 2026

revised September 2026

Abstract

Fedimint issues eCash notes from a federation of mints using blind signatures, which hides *who* spends a note but not *how much* it is worth: every note carries a public denomination. We describe a confidential variant in which the amount is a hidden attribute of a threshold-issued anonymous credential. A note is a Pointcheval–Sanders signature over three hidden attributes (amount, serial secret, spend condition), issued blindly by any t of n mints following the Coconut construction in the Pedersen-commitment form used by Nym; the amount is additionally carried in a Pedersen commitment so that transactions can be balanced homomorphically and range-proven with Bulletproofs. Spending shows a re-randomised signature together with a deterministic nullifier and a short Σ -proof, so that spends are unlinkable to issuance and double spends are detected by nullifier equality. We fix the notation, explain the purpose of every element of the construction, give the security model and proof sketches, describe how notes compose into transactions, and analyse an amortised Σ -proof that keeps the issuance proof at a constant 496 bytes for any number of notes. We show that the amortisation as implemented shares one signature base across a batch and thereby enables an affine-combination forgery of PS signatures; we give a partially amortised variant with per-note bases that avoids it at about 192 bytes per note. We report proof and transaction sizes and timings of a Rust prototype over BLS12-381.

1 Introduction

Electronic cash (eCash) aims to provide a digital analogue of physical cash, enabling users to perform electronic payments while preserving strong privacy guarantees. Traditional eCash systems focus on anonymity by preventing the issuer from linking the withdrawal of a coin to its subsequent spending through the use of blind signatures. More recent constructions extend this notion of privacy by providing *confidential* eCash, where not only the identity of the user remains protected, but also the value of the eCash note. In such systems, the denomination is cryptographically hidden from all parties except those authorized to learn it, while still allowing the correctness of transactions to be publicly verified.

Designing a confidential blind eCash system requires satisfying several fundamental security and practical properties. First, the system should guarantee **unlinkability**, ensuring that an issued note cannot be linked to the transaction in which it is later spent, thereby preserving the user’s anonymity. Second, it must satisfy **unforgeability**, meaning that every valid eCash note must originate from the mint and that no adversary can create additional spendable notes or increase their value. Third, the protocol should support **offline transactions**, allowing payments to be completed without requiring communication with the mint during the spending phase. Finally, the construction should be **practical**, with cryptographic proofs and transaction sizes that remain compact and computational costs that are sufficiently low for deployment in real-world payment systems.

Fedimint today meets the first three goals with blind BLS signatures over a fixed set of denominations [2]: a note is a signature on a random serial, and its value is the public denomination of the key that signed it. Hiding the value therefore needs a different primitive: a signature on a *hidden* amount that can still be verified, balanced against other notes, and bounded to a legal range. Anonymous credentials provide exactly this, and the Coconut construction [9] makes them threshold-issuable, which matches a federation of mints.

Contributions. This paper describes such a confidential federated eCash scheme and the prototype that implements it. Concretely, it

- presents the construction with a complete, typed notation (Section 3) and explains, for every element of the issuance request, the signature and the spend, which security property it exists for (Section 5);
- states the system model, the threat model and the security goals (Section 4) and gives proof sketches reducing each goal to standard assumptions (Section 8);
- describes how notes compose into balanced, range-proven transactions and how double spending is detected (Section 6);
- analyses the amortised issuance proof used by the prototype (Section 7): it proves a soundness bound for the batched Σ -protocol, shows that sharing one signature base across a batch enables an affine-combination forgery of the underlying signatures, and proposes a partially amortised variant with per-note bases that removes the weakness at a modest size cost;
- reports proof sizes, transaction sizes and measured timings of the Rust prototype (Section 9) and lists where the prototype deviates from the description (Section 10).

Section 2 reviews the building blocks and related work, Section 11 collects open problems, and Appendix A develops the Σ -protocol for linear maps that both proofs of the scheme instantiate.

2 Background and Related Work

2.1 Blind-signature eCash and Fedimint

Chaum’s untraceable payments [1] introduced the pattern that all later eCash follows: a user obtains a signature on a blinded message, unblinds it, and later presents the signature, which the issuer cannot connect to the blinded message it signed. Fedimint [2] applies this pattern with threshold BLS signatures [3, 4]: a federation of n guardians holds Shamir shares of a signing key per denomination, a user submits a blinded serial, and any t guardians’ signature shares combine into a note. Values are public because the denomination is the key. A transaction spends notes as inputs and requests new notes as outputs, and the federation checks that the denominations balance. The scheme in this paper keeps the federation, the threshold issuance and the transaction structure, and replaces the per-denomination key by a single key over hidden attributes.

2.2 Pointcheval–Sanders signatures and Coconut

Camenisch–Lysyanskaya signatures [5] made it possible to sign committed messages and to prove possession of a signature in zero knowledge, the basis of anonymous credentials. Pointcheval and Sanders [6] gave a much shorter scheme with the same features in type-3 pairing groups: a signature is two elements of $\mathbb{G}_1$ regardless of the number of messages, signing is linear in both the key and the messages, and signatures can be re-randomised. Its security was first argued under an interactive LRSW-type assumption [8] and later under the non-interactive q -MSDH-1 assumption [7]. Coconut [9] observed that the linearity in the key allows threshold issuance by Shamir sharing and Lagrange interpolation in the exponent, provided all authorities use

the same signature base; it derives that base by hashing a commitment supplied by the user, following the hash-to-base idea of BLS. Coconut carries the hidden attributes to the authorities as ElGamal ciphertexts of $m_j B$.

2.3 Nym’s variant

Nym Technologies deploys Coconut credentials in its network [10] and, for its zk-nym bandwidth credentials, a compact divisible eCash scheme built on the same signature [11, 12]. Their issuance protocol replaces the ElGamal ciphertexts by Pedersen commitments $C_j = m_j B + o_j G_1$ in the hashed base, with the verification key published in both source groups so that the user can strip the blinding term with public information only. This removes the user’s ElGamal key pair and shortens the request. The scheme of this paper follows Nym’s issuance protocol and attribute layout but uses the credential differently at spend time: Nym blinds the attribute term of the verification equation with fresh randomness because it is not a nullifier, whereas here that term is deliberately deterministic (Section 5.8).

2.4 Confidential amounts

Hiding amounts behind Pedersen commitments [13] and balancing a transaction by the homomorphic sum of its commitments is the Confidential Transactions design [14]. Because commitments are over a prime field, a balanced transaction can still create value through wrap-around unless every output is proven to lie in a small range; Bulletproofs [15] provide such range proofs with size logarithmic in the bit length and aggregatable across outputs without a trusted setup. We reuse both ideas and bind the committed amount to the signed attribute with the issuance proof.

2.5 Nullifiers and compact eCash

Detecting double spending of an anonymous token by publishing a deterministic function of a hidden secret, the *nullifier*, goes back to compact eCash [16] and is used by Zcash [17]. Our nullifier $K' = m_1 X_1 + m_2 X_2$ is the hidden-attribute part of the PS verification equation itself, so it costs nothing extra to verify and is tied to the signature by the pairing check rather than by an additional proof.

2.6 Batch verification and amortised proofs

Checking many linear relations at once through a random linear combination is the small-exponents batch verification of Bellare, Garay and Rabin [18]. Applied inside a Σ -protocol it lets a prover answer many instances of the same linear map with one response, which is the amortisation used in Section 7. Logarithmic-size alternatives exist, notably the inner-product folding of Bulletproofs and compressed Σ -protocols [19]; they trade prover and verifier work for size and are not used here.

3 Notation and Setting

3.1 Scalars, groups and the pairing

Throughout, $\mathbb{K} := \mathbb{Z}/p\mathbb{Z}$ is the field of integers modulo the prime p , the order of the BLS12-381 groups. Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be the BLS12-381 pairing groups, all of order p and *written additively*, and let

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$$

be the pairing. It is bilinear: for all $a, b \in \mathbb{K}$, $U \in \mathbb{G}_1$ and $W \in \mathbb{G}_2$,

$$e(aU, bW) = ab e(U, W).$$

Since each group has prime order p , scalar multiplication $\mathbb{K} \times \mathbb{G}_i \rightarrow \mathbb{G}_i$, $(\lambda, U) \mapsto \lambda U$ makes every $\mathbb{G}_i$ a one-dimensional $\mathbb{K}$ -vector space. All maps in this document that send attributes and randomness to group elements are therefore $\mathbb{K}$ -linear maps, and the whole scheme can be read in the language of linear algebra. There is no efficient map from $\mathbb{G}_1$ to $\mathbb{G}_2$ or back (type-3 pairing), which is what makes it safe to publish key material in both groups.

Colour legend: every symbol is coloured by the set it lives in

$m, r, x, \rho, s, \gamma, \lambda$	scalars in $\mathbb{K}$ (32 B)	lowercase
$G_1, G_p, H_p, H_j, B, C_j, P, Y_j, \sigma, R$	elements of $\mathbb{G}_1$ (48 B)	uppercase or Greek
G_2, X_j, K', V	elements of $\mathbb{G}_2$ (96 B)	uppercase
$e(\cdot, \cdot)$	elements of $\mathbb{G}_T$	only inside pairing checks

Groups are written additively. Group elements are uppercase letters (or Greek, as for the signature σ); scalars are lowercase. Sizes are for compressed encodings. A scalar times a $\mathbb{G}_1$ element is a $\mathbb{G}_1$ element; a scalar times a $\mathbb{G}_2$ element is a $\mathbb{G}_2$ element; elements of different groups are never added to each other. Vectors are written with an arrow ($\vec{w}$, $\vec{C}$) and are typed component-wise.

3.2 Generators and hash functions

$G_1 \in \mathbb{G}_1$ and $G_2 \in \mathbb{G}_2$ are fixed generators. All further generators are derived by hashing domain-separation strings to the curve (nothing-up-my-sleeve), so that nobody knows a discrete logarithm relation between any two of them:

- $G_p, H_p \in \mathbb{G}_1$ – Pedersen basis for *amounts*;
- $H_1, H_2, H_3 \in \mathbb{G}_1$ – basis for the attribute commitment C_m .

The generator G_1 doubles as the blinding generator of all commitments that the mint signs. This is not a coincidence: unblinding in Section 5.6 only works because the blinding generator of the commitments is the same generator in which the mint’s public key is published in $\mathbb{G}_1$.

Two hash functions, modelled as random oracles, are used:

$$H_{G_1} : \{0, 1\}^* \rightarrow \mathbb{G}_1 \quad \text{and} \quad H_s : \{0, 1\}^* \rightarrow \mathbb{K}.$$

H_{G_1} derives the per-note signature base B ; H_s derives Fiat–Shamir challenges and encodes the spend condition as a scalar. $\parallel$ denotes concatenation of canonical encodings.

3.3 Attributes

Each eCash note carries three attributes in $\mathbb{K}$. They are the messages that the federation signs, and they are never revealed to the mint at issuance.

Attribute	Meaning	Origin
m_1	the <i>amount</i>	a 64-bit integer lifted to $\mathbb{K}$
m_2	the <i>serial secret</i>	uniformly random; basis of the nullifier
$m_3 = H_s(\text{auth})$	the <i>spend condition</i>	hash of authentication data fixed at issuance

3.4 Keys

For a single issuer, the secret key is a vector of four scalars,

$$\text{sk} = (x_0, x_1, x_2, x_3) \in \mathbb{K}^4,$$

one *constant term* x_0 plus one scalar x_j per attribute m_j . The public key publishes each component in *both* source groups:

$$\text{pk} = \left(\underbrace{\{Y_j = x_j G_1\}_{j=0}^3}_{\text{in } \mathbb{G}_1: \text{ used for unblinding}}, \underbrace{\{X_j = x_j G_2\}_{j=0}^3}_{\text{in } \mathbb{G}_2: \text{ used for verification}} \right).$$

In Coconut’s notation X_0 is α , X_j is β_j and Y_j is Nym’s $\beta_{g_1,j}$ [12]. Section 5.7 replaces the single issuer by a t -of- n federation whose *aggregate* key has exactly this shape.

3.5 Assumptions

The security arguments of Section 8 rely on the following standard assumptions, all believed to hold in BLS12-381.

DL. The discrete-logarithm problem is hard in $\mathbb{G}_1$ and $\mathbb{G}_2$. It makes the Pedersen commitments binding: an opening of P to two different amounts yields the discrete logarithm of H_p to base G_p .

DDH / SXDH. Decisional Diffie–Hellman is hard in $\mathbb{G}_1$ and in $\mathbb{G}_2$ separately (the symmetric external Diffie–Hellman assumption), which is plausible precisely because the pairing is type-3. It makes re-randomised signatures unlinkable and hides m_1 inside K' .

PS unforgeability. Pointcheval–Sanders signatures are existentially unforgeable under chosen-message attack. This holds under the interactive LRSW-type assumption of [6] or, for signatures with q messages, under the q -MSDH-1 assumption of [7]. We note in Section 7.3 that the assumption presupposes a fresh base per signature.

Random oracles. $H_{\mathbb{G}_1}$ and H_s are modelled as random oracles. $H_{\mathbb{G}_1}$ gives every note a base with unknown discrete logarithm; H_s makes the Fiat–Shamir transform sound.

3.6 Summary of symbols

Table 1 lists every symbol used in the remainder of the document together with the set it lives in.

4 System Model and Security Goals

4.1 Parties and threat model

There are n *mints* (Fedimint guardians), each holding a share of the federation’s signing key, and any number of *users*. A mint acts both as an issuer, when it signs a blinded request, and as a verifier, when it checks a spend as part of validating a transaction. Mints run a consensus protocol; we treat it as a black box that delivers the same transactions in the same order to every honest mint and bounds the size of a transaction.

An adversary may corrupt up to $t - 1$ mints and any number of users, adaptively but statically within one issuance. Corrupted parties collude and share all state, including key shares. Honest mints run the protocol as specified and keep their key shares secret. The network is under adversarial control; all messages between users and mints are visible to the adversary. We do not model denial of service.

4.2 Syntax

The scheme consists of the following algorithms, which Section 5 instantiates.

KeyGen(t, n) outputs an aggregate public key pk , public key shares $\text{pk}_1, \dots, \text{pk}_n$ and secret key shares $\text{sk}_1, \dots, \text{sk}_n$.

Request(v, auth) run by a user with amount v and spend condition auth ; outputs a public request req and private state st .

Table 1: Symbols, their types and their meaning.

Symbol	Type	Meaning
$p, \mathbb{K}$	–	group order and scalar field $\mathbb{Z}/p\mathbb{Z}$
G_1, G_2	$\mathbb{G}_1, \mathbb{G}_2$	fixed generators (Coconut's g_1, g_2); G_1 is also the blinding generator
G_p, H_p	$\mathbb{G}_1$	Pedersen basis for amounts
H_1, H_2, H_3	$\mathbb{G}_1$	basis for the attribute commitment
m_1, m_2, m_3	$\mathbb{K}$	amount, serial secret, spend condition
auth	bytes	authentication data with $m_3 = H_s(\text{auth})$
r_p, r_m, r_1, r_2, r_3	$\mathbb{K}$	commitment randomness chosen by the user at issuance
P	$\mathbb{G}_1$	Pedersen commitment to the amount
C_m	$\mathbb{G}_1$	commitment to all attributes; input to $H_{\mathbb{G}_1}$
B	$\mathbb{G}_1$	signature base, $B = H_{\mathbb{G}_1}(C_m)$ (Coconut's h)
C_1, C_2, C_3	$\mathbb{G}_1$	blinded attributes (commitments in base B)
$\vec{C}$	$\mathbb{G}_1^5$	the statement of the issuance proof, (P, C_m, C_1, C_2, C_3)
$\vec{w}$	$\mathbb{K}^8$	the witness of the issuance proof, $(m_1, m_2, m_3, r_p, r_m, r_1, r_2, r_3)$
φ_B	$\mathbb{K}^8 \rightarrow \mathbb{G}_1^5$	linear map with $\varphi_B(\vec{w}) = \vec{C}$
$\vec{\rho}, \vec{R}, \gamma, \vec{s}$	$\mathbb{K}^8, \mathbb{G}_1^5, \mathbb{K}, \mathbb{K}^8$	Σ -protocol nonce, commitment, challenge, response
$x_0, \dots, x_3$	$\mathbb{K}$	secret key
$Y_j = x_j G_1, X_j = x_j G_2$	$\mathbb{G}_1, \mathbb{G}_2$	public key in both groups
$\tilde{\sigma}, \sigma$	$\mathbb{G}_1$	blinded and unblinded signature; (B, σ) is the credential
λ_i, S	$\mathbb{K}, \text{set}$	Lagrange coefficients and the set of t contributing mints
r, r', r'_p	$\mathbb{K}$	re-randomisation and fresh commitment randomness at spend time
(B', σ', V)	$\mathbb{G}_1, \mathbb{G}_1, \mathbb{G}_2$	re-randomised signature shown at spend time
P'	$\mathbb{G}_1$	fresh Pedersen commitment to the amount at spend time
K'	$\mathbb{G}_2$	nullifier, $K' = m_1 X_1 + m_2 X_2$
ψ	$\mathbb{K}^3 \rightarrow \mathbb{G}_1 \times \mathbb{G}_2$	linear map of the spend proof

$\text{Sign}(\text{sk}_i, \text{req})$ run by mint i ; verifies req and outputs a blinded signature share $\tilde{\sigma}_i$ or $\perp$.
 $\text{Finalize}(\text{st}, \{\tilde{\sigma}_i\}_{i \in S})$ run by the user with shares from a set S of t mints; outputs a note note or $\perp$.
 $\text{Spend}(\text{note})$ outputs a spend sp and the nullifier K' .
 $\text{Verify}(\text{pk}, \text{sp}, \text{auth})$ outputs 0 or 1.
 Amount commitments appear in both req and sp so that the transaction layer (Section 6) can balance them.

4.3 Security goals

Correctness. A note obtained from t honest mints on an honest request verifies, and its spend reveals the nullifier of that note.
Unforgeability (balance). No adversary corrupting fewer than t mints can produce k valid spends with pairwise distinct nullifiers whose committed amounts sum to more than the amounts of the notes issued to it by the honest mints. This covers creating notes from nothing, increasing a note's value, and spending a note more than once.
Blindness. The mints learn nothing about the attributes (m_1, m_2, m_3) from an issuance beyond what the range proof states.
Unlinkability. Given two spends and two issuance transcripts, an adversary, even one holding all key shares, cannot tell which spend came from which issuance better than by guessing.
Double-spend detection. Two spends of the same note have the same nullifier; two spends of different notes have different nullifiers except with negligible probability.
Value confidentiality. An observer of all transcripts learns nothing about a note's amount beyond the range proven at issuance and the balance relations of the transactions it appears in.

5 The Scheme for a Single Note

This section presents the lifecycle of *one* eCash note, first with a single issuer and then with a federation of mints. Batching several notes into one request is deferred to Section 7; nothing in this section depends on it.

5.1 The underlying signature

A Pointcheval–Sanders signature [6] on the attributes (m_1, m_2, m_3) with base point $B \in \mathbb{G}_1$ (Coconut's h) is the pair (B, σ) with

$$\sigma = \left(x_0 + \sum_{j=1}^3 x_j m_j \right) B, \quad (1)$$

verified with the public key in $\mathbb{G}_2$ by the single pairing equation

$$e\left(B, X_0 + \sum_{j=1}^3 m_j X_j\right) \stackrel{?}{=} e(\sigma, G_2). \quad (2)$$

Correctness is bilinearity: both sides equal $e(B, G_2)$ raised to $x_0 + \sum_j x_j m_j$. Three properties of this signature carry the whole scheme.

Why PS signatures

1. **Signing is linear in the key.** Equation (1) is a linear function of $(x_0, \dots, x_3)$. Anything linear in the key can be Shamir-shared and interpolated in the exponent, which gives

threshold issuance (Section 5.7).

2. **Signing is linear in the messages.** A commitment $m_j B + r_j G_1$ can be fed into (1) in place of $m_j B$; the signer never sees m_j , and the extra term is removable. This gives blind issuance (Section 5.5).
3. **Re-randomisable.** For any $r \neq 0$, $(rB, r\sigma)$ is an equally valid signature on the same messages, and is unlinkable to (B, σ) under DDH in $\mathbb{G}_1$. This gives unlinkable spending (Section 5.8).

The base B must be different for every signature. If two signatures share a base, their affine combinations are again valid signatures, on the corresponding affine combination of the messages (Coconut, footnote 3 in [9]; see also Section 7.3). In plain PS the signer picks B at random, which a set of independent mints cannot do consistently; Coconut instead derives B by hashing a commitment supplied by the user, so that every mint computes the same B on its own.

5.2 Requesting issuance

The user holds the attributes (m_1, m_2, m_3) , samples the randomness r_p, r_m, r_1, r_2, r_3 uniformly from $\mathbb{K}$, and forms five elements of $\mathbb{G}_1$:

$$P = m_1 G_p + r_p H_p \quad (\text{Pedersen commitment to the amount}) \quad (3)$$

$$C_m = m_1 H_1 + m_2 H_2 + m_3 H_3 + r_m G_1 \quad (\text{commitment to all attributes}) \quad (4)$$

$$B = H_{\mathbb{G}_1}(C_m) \quad (\text{derived signature base}) \quad (5)$$

$$C_j = m_j B + r_j G_1 \quad (\text{blinded attributes, } j \in \{1, 2, 3\}) \quad (6)$$

The issuance request consists of the statement $\vec{C} = (P, C_m, C_1, C_2, C_3) \in \mathbb{G}_1^5$, the proof of Section 5.3, and the range proof of Section 5.4. The mint recomputes B from C_m itself; it is not transmitted.

What each element is for

P – the amount commitment. Pedersen commitment in its own basis (G_p, H_p) : perfectly hiding because of r_p , binding under the discrete-logarithm assumption between G_p and H_p . It is *additively homomorphic*, $P(m, r) + P(m', r') = P(m + m', r + r')$, which lets the transaction layer check that inputs and outputs balance without seeing the amounts. It is also the object the range proof (Section 5.4) is attached to. It is deliberately in a basis unrelated to the signature so that balance checking is independent of the credential.

C_m – the commitment behind B . A Pedersen vector commitment to all three attributes with randomness r_m . Its only purpose is to be hashed into B . Because it is *binding*, B is a function of the attributes the user is about to get signed; because it is *hiding*, the mint learns nothing from it; because r_m is fresh, every request gets a fresh base even for identical attributes.

B – the signature base. Neither party chooses it: the user cannot pick a base with a known relation to another note's base (which would enable the affine-combination forgery of Section 5.1), and the mint does not have to pick anything, so all mints of a federation agree on B without communicating. This is the Coconut trick that turns PS into a threshold-friendly scheme.

C_j – the blinded attributes. Pedersen commitments to each attribute *in the base B* with blinding on G_1 . The base B is what the signature (1) multiplies the attributes with, so the mint can apply its key directly to C_j (Section 5.5). The blinding generator G_1 is what the public key Y_j is published in, so the blinding term the mint inadvertently signs,

$r_j x_j G_1 = r_j Y_j$, is computable by the user from public data. In the original Coconut these were ElGamal ciphertexts of $m_j B$; Nym's commitments serve the same purpose with less data.

5.3 The issuance proof

The mint must be convinced that $\vec{C}$ is well formed before signing: otherwise a user could, for instance, put one amount into P (and its range proof) and a different amount into C_1 (and the signature). Well-formedness of $\vec{C}$ is exactly the statement that $\vec{C}$ lies in the image of the linear map $\varphi_B : \mathbb{K}^8 \rightarrow \mathbb{G}_1^5$,

$$\varphi_B(m_1, m_2, m_3, r_p, r_m, r_1, r_2, r_3) = \begin{pmatrix} m_1 G_p + r_p H_p \\ m_1 H_1 + m_2 H_2 + m_3 H_3 + r_m G_1 \\ m_1 B + r_1 G_1 \\ m_2 B + r_2 G_1 \\ m_3 B + r_3 G_1 \end{pmatrix} = \begin{pmatrix} P \\ C_m \\ C_1 \\ C_2 \\ C_3 \end{pmatrix} = \vec{C}. \quad (7)$$

The witness, known only to the user and never transmitted, is

$$\vec{w} = (m_1, m_2, m_3, r_p, r_m, r_1, r_2, r_3) \in \mathbb{K}^8.$$

Note that φ_B depends on B , which the verifier recomputes from the second component of $\vec{C}$; the map is fixed once C_m is.

The user proves knowledge of a preimage with the standard Σ -protocol for linear maps (Appendix A; Schnorr's protocol generalised to a vector-valued linear map), made non-interactive with the Fiat–Shamir transform under the domain tag $\text{tag}_{\text{iss}} = \text{FEDIMINT_ECASH_CHALLENGE_ISSUANCE}$:

$$\begin{aligned} \vec{\rho} &\xleftarrow{\$} \mathbb{K}^8, & \vec{R} &= \varphi_B(\vec{\rho}) \in \mathbb{G}_1^5, \\ \gamma &= H_s(\text{tag}_{\text{iss}} \parallel \vec{C} \parallel \vec{R}) \in \mathbb{K}, & \vec{s} &= \vec{\rho} + \gamma \vec{w} \in \mathbb{K}^8. \end{aligned} \quad (8)$$

The proof is $(\vec{R}, \vec{s})$: five $\mathbb{G}_1$ elements and eight scalars, $5 \cdot 48 + 8 \cdot 32 = 496$ bytes. The verifier recomputes B and γ and checks

$$\varphi_B(\vec{s}) \stackrel{?}{=} \gamma \vec{C} + \vec{R}. \quad (9)$$

Why this proves what we need

Completeness. By linearity, $\varphi_B(\vec{s}) = \varphi_B(\vec{\rho}) + \gamma \varphi_B(\vec{w}) = \vec{R} + \gamma \vec{C}$.

Knowledge (special soundness). Two accepting transcripts with the same $\vec{R}$ and different challenges $\gamma \neq \gamma'$ give $\varphi_B(\vec{s} - \vec{s}') = (\gamma - \gamma') \vec{C}$, so $(\gamma - \gamma')^{-1}(\vec{s} - \vec{s}')$ is a witness. A prover who can answer two challenges knows a preimage. In the random-oracle model Fiat–Shamir preserves this.

Zero knowledge. A simulator picks $\vec{s} \xleftarrow{\$} \mathbb{K}^8$ and $\gamma \xleftarrow{\$} \mathbb{K}$ first and sets $\vec{R} = \varphi_B(\vec{s}) - \gamma \vec{C}$; the transcript is distributed exactly like a real one. The mint learns nothing about the attributes beyond the commitments, which are hiding.

What the single witness buys us. One witness vector serves all five equations, so the same m_1 sits in P , C_m and C_1 , and the same m_2, m_3 sit in C_m and C_2, C_3 . Hence (i) the amount that is range-proven is the amount that gets signed (unforgeability of value), (ii) B really is derived from a commitment to the attributes being signed (the freshness argument for B holds), and (iii) the C_j are of the exact form (6), so the blinding term in the signature is exactly $\sum_j r_j Y_j$ and nothing else gets signed. Hashing $\vec{C}$ into the challenge binds the proof to the statement; the domain tag prevents replaying a proof

for ψ of Section 5.8 against φ_B .

5.4 Range proof on the amount

The commitment P hides m_1 as an element of $\mathbb{K}$, and $\mathbb{K}$ wraps around: a “negative” amount $-v$ is the field element $p - v$. Without further constraints an adversary could commit to an out-of-range value and satisfy a balance equation $\sum P_{\text{in}} = \sum P_{\text{out}}$ while creating value from nothing. The user therefore additionally proves that the committed amount lies in an accepted interval, e.g. $0 \leq m_1 < 2^{64}$. This work employs Bulletproofs [15], which give short non-interactive range proofs over Pedersen commitments without a trusted setup, with proof size logarithmic in the bit length and aggregatable across several commitments. Because the issuance proof ties the m_1 in P to the m_1 in the signature, the range proof on P carries over to the signed amount.

5.5 Blind signing

A mint that has verified the proofs recomputes $B = H_{\mathbb{G}_1}(C_m)$ and applies its key to the request as if the C_j were the attributes:

$$\tilde{\sigma} = x_0 B + \sum_{j=1}^3 x_j C_j = \underbrace{\left(x_0 + \sum_{j=1}^3 x_j m_j\right) B}_{\text{PS signature } \sigma, \text{ eq. (1)}} + \underbrace{\sum_{j=1}^3 r_j (x_j G_1)}_{\text{blinding term} = \sum_j r_j Y_j}. \quad (10)$$

The expansion is just (6) substituted into the first expression. The mint never sees m_j ; it sees B , the C_j and the proof, all of which are independent of the attributes by the hiding property and zero knowledge.

5.6 Unblinding and verification

The blinding term in (10) is a linear combination of the $\mathbb{G}_1$ public key components with coefficients the user chose, so the user removes it from public information only:

$$\sigma = \tilde{\sigma} - \sum_{j=1}^3 r_j Y_j, \quad (11)$$

and then checks (B, σ) against the $\mathbb{G}_2$ key with the PS equation (2). This is where the choice of G_1 as the blinding generator in (6) is essential: with any other blinding generator the user would need x_j to cancel the term. The pair (B, σ) together with (m_1, m_2, auth) is the eCash note.

5.7 Threshold issuance

Nothing above requires a single issuer, because (10) is $\mathbb{K}$ -linear in the secret key $(x_0, \dots, x_3)$. The standard threshold-BLS transformation [4] therefore applies component-wise, with Shamir sharing [21] of each key component.

Key sharing. Each x_j is Shamir-shared: pick random polynomials $f_j \in \mathbb{K}[X]$ of degree at most $t - 1$ with $f_j(0) = x_j$, and give mint $i \in \{1, \dots, n\}$ the key share

$$\text{sk}_i = (f_0(i), f_1(i), f_2(i), f_3(i)) \in \mathbb{K}^4,$$

with public key shares $Y_{i,j} = f_j(i) G_1$ and $X_{i,j} = f_j(i) G_2$. The aggregate public key is $Y_j = f_j(0) G_1$, $X_j = f_j(0) G_2$, i.e. exactly the single-issuer key of Section 3. Fewer than t shares reveal nothing about x_j .

Share signing and share verification. Every mint runs Section 5.5 with its own share,

$$\tilde{\sigma}_i = f_0(i) B + \sum_{j=1}^3 f_j(i) C_j,$$

and the user unblinds each share with the mint's $\mathbb{G}_1$ key share, $\sigma_i = \tilde{\sigma}_i - \sum_j r_j Y_{i,j}$, then verifies it with (2) against $X_{i,j}$. A mint that returns garbage is identified immediately and simply left out.

Aggregation. Let S be a set of t mint indices whose shares verified. The federation signature is the Lagrange combination at 0,

$$\sigma = \sum_{i \in S} \lambda_i \sigma_i, \quad \lambda_i = \prod_{\substack{k \in S \\ k \neq i}} \frac{k}{k-i} \pmod{p}. \quad (12)$$

Since $\sigma_i = (f_0(i) + \sum_j m_j f_j(i)) B$ and $\lambda \mapsto \lambda B$ is linear, the sum evaluates the polynomial $f_0 + \sum_j m_j f_j$ at 0 inside the group, which is exactly the single-issuer signature (1). Everything downstream is unchanged and verifies against the aggregate key.

5.8 Spending

To spend, the user must show the mint a valid signature on a note without letting the mint connect it to the issuance it came from, while still letting the mint (i) check the amount against the transaction balance and (ii) detect a second spend of the same note.

Re-randomisation. The user samples $r, r' \xleftarrow{\$} \mathbb{K}$ and forms

$$(B', \sigma', V) = (r B, r \sigma + r r' B, r' G_2). \quad (13)$$

The pair $(r B, r \sigma)$ is the PS re-randomisation of Section 5.1: fresh group elements, unlinkable to (B, σ) under DDH in $\mathbb{G}_1$. The r' terms are inherited from Coconut's (κ, ν) and are discussed in Section 5.8.1.

Shown elements. Along with (B', σ', V) and the authentication data `auth` the user sends

$$P' = m_1 G_p + r'_p H_p \in \mathbb{G}_1, \quad K' = m_1 X_1 + m_2 X_2 \in \mathbb{G}_2, \quad (14)$$

where $r'_p \xleftarrow{\$} \mathbb{K}$ is *fresh* commitment randomness, together with a second Fiat–Shamir Σ -proof (tag `tagsp` = `FEDIMINT_ECASH_CHALLENGE_SPEND`) of knowledge of (m_1, m_2, r'_p) for the linear map

$$\psi : \mathbb{K}^3 \rightarrow \mathbb{G}_1 \times \mathbb{G}_2, \quad \psi(m_1, m_2, r'_p) = (m_1 G_p + r'_p H_p, m_1 X_1 + m_2 X_2) = (P', K'). \quad (15)$$

The protocol is the same as in (8) with ψ in place of φ_B and witness (m_1, m_2, r'_p) ; the prototype transmits the challenge instead of the commitment $(R_{P'}, R_K)$, so a spend proof is four scalars (128 bytes), and the verifier recomputes the commitment as $\psi(\vec{s}) - \gamma(P', K')$.

Verification. The mint recomputes $m_3 = H_s(\text{auth})$, requires $m_3 \neq 0$ and $B' \neq 0$, verifies the ψ -proof, checks that K' has not been seen before, and finally checks the signature:

$$e(B', X_0 + K' + m_3 X_3 + V) \stackrel{?}{=} e(\sigma', G_2). \quad (16)$$

Expanding both sides shows why it holds. Write $z := x_0 + x_1 m_1 + x_2 m_2 + x_3 m_3$, so that $\sigma = z B$ and $X_0 + K' + m_3 X_3 = z G_2$. Then

$$\begin{aligned} \text{LHS} &= e(rB, (z + r') G_2) = r(z + r') e(B, G_2), \\ \text{RHS} &= e(rz B + rr' B, G_2) = r(z + r') e(B, G_2). \end{aligned}$$

The equation is the PS verification (2) with the hidden part $m_1 X_1 + m_2 X_2$ of the message supplied as the single element K' , the revealed part $m_3 X_3$ recomputed by the mint, and the r' terms cancelling on both sides.

What each shown element achieves

(B', σ') – **unforgeability, unlinkability.** A valid re-randomised PS signature proves the federation signed *some* attributes; fresh randomness makes the pair independent of the issuance transcript.

K' – **the nullifier and the hidden message.** It supplies the hidden attributes to the verification equation without revealing them, and it is *deterministic* in (m_1, m_2) : spending the same note twice produces the same K' , so the mints keep a set of seen K' and reject repeats. The random m_2 makes K' unique per note and, since m_2 was only ever seen blinded, unlinkable to issuance. Its soundness rests on the ψ -proof: a spender must *know* (m_1, m_2) behind K' , so K' cannot be re-randomised into a fresh nullifier for the same note.

P' – **confidential amount at spend time.** A fresh Pedersen commitment to the same amount, in the balance-check basis, with new randomness so it is unlinkable to P . The ψ -proof ties the m_1 in P' to the m_1 in K' , and (16) ties the m_1 in K' to the signature. So the amount the transaction layer balances with is the amount the federation signed, which is the amount that was range-proven at issuance.

m_3, auth – **the spend condition.** m_3 is baked into σ , so only a spender who can present the committed auth makes (16) hold. Revealing auth does not link the spend to issuance because the mint only saw m_3 blinded. Requiring $m_3 \neq 0$ excludes the degenerate case in which the third attribute contributes nothing to the signature.

5.8.1 On the role of V

In Coconut the spender shows $\kappa = X_0 + \sum_j m_j X_j + r' G_2$ and $\nu = r' B'$, and the verifier checks $e(B', \kappa) = e(\sigma' + \nu, G_2)$. The fresh r' in κ is what hides the attributes there. In this scheme $V = r' G_2$ plays the part of κ 's blinding term and $rr' B$ inside σ' plays the part of ν . However, K' is deliberately *not* blinded, because it has to be a deterministic nullifier; and $e(B', V) = e(rr' B, G_2)$ is computable by anyone from the shown data, so an observer can cancel the r' contribution from $e(\sigma', G_2)$ and recover the same $\mathbb{G}_T$ values as if $r' = 0$. Under this reading V adds 96 bytes and one scalar multiplication but no additional hiding; the unlinkability of a spend rests on the re-randomisation by r and on the fresh r'_p . Whether V should be kept is an open question (Section 11).

5.9 Summary of properties

Table 2 maps each goal of Section 4 to the elements that provide it; Section 8 gives the arguments.

Table 2: Where each security property comes from.

Property	Where it comes from
Unforgeability	Existential unforgeability of PS signatures (LRSW-type assumption [6]) with a per-note base $B = H_{\mathbb{G}_1}(C_m)$; binding of P , C_m , C_j ; soundness of the φ_B -proof (same m_1 everywhere) and of the ψ -proof; the range proof on P . Fewer than t mints cannot sign.
Blindness	The mint sees only hiding commitments $\vec{C}$ and zero-knowledge proofs; the blinded signature reveals nothing further.
Unlinkability	Re-randomisation ($rB, r\sigma$) (DDH in $\mathbb{G}_1$) and fresh r'_p in P' make the spend transcript independent of the issuance transcript; m_2 makes K' independent of anything shown at issuance.
Double-spend detection	Equality of nullifiers K' , with the ψ -proof preventing a spender from producing a second nullifier for the same (m_1, m_2) .
Value confidentiality	P and P' are perfectly hiding; K' hides m_1 under DDH in $\mathbb{G}_2$ because of the random m_2 .
Offline spending	Verification (16) needs only the aggregate public key; only double-spend detection needs the mints' shared state.

6 Transactions

A note on its own is a credential; eCash arises when notes are spent and re-issued inside transactions validated by the federation. This section describes the intended transaction layer. It is not part of the prototype, but the scheme's elements were chosen for it.

6.1 Structure and balance

A transaction has a inputs, each a spend sp_k of Section 5.8 carrying an amount commitment P'_k , and b outputs, each an issuance request of Section 5.2 carrying an amount commitment P_ℓ , together with a public fee $f \geq 0$. The spender additionally reveals the *excess blinding*

$$\delta = \sum_{k=1}^a r'_{p,k} - \sum_{\ell=1}^b r_{p,\ell} \in \mathbb{K}, \quad (17)$$

and the mints check

$$\sum_{k=1}^a P'_k \stackrel{?}{=} \sum_{\ell=1}^b P_\ell + f G_p + \delta H_p. \quad (18)$$

By the homomorphism of Pedersen commitments, (18) holds if and only if $\sum_k m_{1,k} = \sum_\ell m_{1,\ell} + f$ in $\mathbb{K}$ or the spender has found a discrete-logarithm relation between G_p and H_p . Revealing δ leaks nothing about the individual amounts because each $r_{p,\ell}$ is uniform. Peg-ins and peg-outs are the cases $a = 0$ and $b = 0$ respectively, with the on-chain amount taking the place of f as a public term; their Bitcoin-side handling is unchanged from Fedimint and not discussed here.

6.2 Range proofs

Equation (18) is over $\mathbb{K}$, so it is satisfied by an output committing to 2^{64} and another committing to $p - 2^{64} + v$. Every output therefore carries a Bulletproof that its amount lies in $[0, 2^{64})$; the b proofs aggregate into one of size $O(\log b)$. Inputs need no range proof: their amounts

were range-proven when they were issued, and the issuance proof (7) ties the range-proven commitment to the signed attribute, while the spend proof (15) and the pairing check (16) tie the signed attribute to P'_k . This chain is exactly what the shared-base weakness of Section 7.3 breaks, because it lets a user obtain a signature on an amount that was never range-proven.

6.3 Validation and the nullifier set

To validate a transaction, every mint

1. verifies each input with `Verify`, i.e. the ψ -proof and the pairing check (16);
2. checks that no input nullifier K'_k is in the set N of nullifiers seen so far, and that the a input nullifiers are pairwise distinct;
3. verifies the issuance proof and the range proof of the outputs;
4. checks the balance (18).

If all checks pass the transaction is accepted by consensus, every mint adds the input nullifiers to N and signs the outputs with its key share. The set N grows by 96 bytes per spent note and is the only state the scheme needs beyond the key shares; it can be pruned once a key is retired.

6.4 Offline spending

Steps 1, 3 and 4 need only the aggregate public key, so a payee can check a payment without the federation. Only step 2 needs the shared state N , so an offline payee accepts the risk that the same note is spent elsewhere before the transaction reaches the federation. This is the usual trade-off of offline eCash; the scheme does not identify double spenders after the fact, it only rejects the second spend.

7 Batching Issuance Requests

A naive approach to requesting several notes at once repeats Section 5.2 per note. The proof alone is then 496 bytes per note, and together with Fedimint's consensus message limit of 64 KB this severely restricts how many notes a single transaction can issue. Several techniques exist to combine multiple Σ -protocol instances into one compact proof: folding as used by Bulletproofs for non-linear relations, or amortisation for parallel instances of the same linear relation. The latter is used here. There are still open discussions concerning the security of the protocol when batching is applied, both for the proofs and for the credential scheme itself; see Section 7.3.

7.1 Batched issuance proof

Suppose the user requests n notes, $i = 1, \dots, n$, with witnesses $\vec{w}_i \in \mathbb{K}^8$ and statements

$$\vec{C}_i = (P_i, C_{m,i}, C_{1,i}, C_{2,i}, C_{3,i}) \in \mathbb{G}_1^5.$$

The amortised protocol sums n instances of (8) into a single response, which only works if all instances use the *same* linear map. Since φ_B depends on B , the batch shares one base derived from all attribute commitments of the batch:

$$B = H_{\mathbb{G}_1}(C_{m,1} \parallel C_{m,2} \parallel \dots \parallel C_{m,n}), \quad \varphi_B(\vec{w}_i) = \vec{C}_i \quad \text{for all } i. \quad (19)$$

For $n = 1$ this coincides with (5). The prover samples one nonce vector per instance, commits to their sum, derives one challenge *per instance*, and sums the responses:

$$\vec{\rho}_i \xleftarrow{\$} \mathbb{K}^8, \quad \vec{R}_i = \varphi_B(\vec{\rho}_i), \quad \vec{R} = \sum_{i=1}^n \vec{R}_i \in \mathbb{G}_1^5, \quad (20)$$

$$\gamma_i = H_s(\text{tag}_{\text{iss}} \parallel \vec{C}_1 \parallel \dots \parallel \vec{C}_n \parallel \vec{R} \parallel i) \in \mathbb{K}, \quad (21)$$

$$\vec{s} = \sum_{i=1}^n (\vec{\rho}_i + \gamma_i \vec{w}_i) \in \mathbb{K}^8. \quad (22)$$

The prover sends $(\vec{R}, \vec{s})$, again 496 bytes regardless of n , along with the statements $\vec{C}_1, \dots, \vec{C}_n$. The verifier recomputes B from (19) and the γ_i from (21) and checks

$$\varphi_B(\vec{s}) \stackrel{?}{=} \sum_{i=1}^n \gamma_i \vec{C}_i + \vec{R}. \quad (23)$$

Why the batched proof works

Completeness. Linearity of φ_B applied to (22): $\varphi_B(\vec{s}) = \sum_i \varphi_B(\vec{\rho}_i) + \sum_i \gamma_i \varphi_B(\vec{w}_i) = \vec{R} + \sum_i \gamma_i \vec{C}_i$. This step is exactly where all instances must share φ_B , i.e. share B .

Soundness. Lemma 1 below: if any $\vec{C}_i$ is not in the image of φ_B , the check (23) passes with probability at most $1/p$ over the choice of γ_i alone. Including the index i in the hash gives each instance its own independent challenge, which the lemma needs.

Zero knowledge. Unchanged: $\vec{s}$ is a sum of uniformly random $\vec{\rho}_i$ plus witness terms, hence uniform.

Lemma 1 (Soundness of the batched check). *Let $\varphi : \mathbb{K}^8 \rightarrow \mathbb{G}_1^5$ be linear, let $\vec{C}_1, \dots, \vec{C}_n \in \mathbb{G}_1^5$ and $\vec{R} \in \mathbb{G}_1^5$ be fixed, and let $\gamma_1, \dots, \gamma_n$ be independent and uniform in $\mathbb{K}$. If $\vec{C}_j \notin \text{im } \varphi$ for some j , then the probability that there exists any $\vec{s} \in \mathbb{K}^8$ with $\varphi(\vec{s}) = \sum_i \gamma_i \vec{C}_i + \vec{R}$ is at most $1/p$.*

Proof. Fix all γ_i with $i \neq j$ and write $\vec{D} := \vec{R} + \sum_{i \neq j} \gamma_i \vec{C}_i$. A response exists iff $\vec{D} + \gamma_j \vec{C}_j \in \text{im } \varphi$. If this held for two values $\gamma_j \neq \gamma'_j$, then, $\text{im } \varphi$ being a subspace, $(\gamma_j - \gamma'_j) \vec{C}_j \in \text{im } \varphi$ and hence $\vec{C}_j \in \text{im } \varphi$, a contradiction. So at most one value of γ_j admits a response, and it is hit with probability $1/p$. $\square$

In the random-oracle model the γ_i of (21) are independent and uniform once $\vec{C}_1, \dots, \vec{C}_n$ and $\vec{R}$ are fixed, so a prover who tries q hash queries succeeds on a false statement with probability at most q/p . Knowledge of each individual witness follows from $n + 1$ accepting transcripts with affinely independent challenge vectors, which a forking argument extracts; the linear system they define is triangular and yields a preimage of every $\vec{C}_i$.

Range proofs batch independently: Bulletproofs aggregate the n commitments $P_1, \dots, P_n$ into one proof of size logarithmic in n .

7.2 Blind signing a batch

Each mint verifies the batched proof once and then signs every note of the batch with the *shared* base, $\tilde{\sigma}_i = x_0 B + \sum_j x_j C_{j,i}$, exactly as in (10); unblinding, share verification, aggregation and spending are per note and unchanged from Section 5. Note that after Section 5.8 the notes of one batch are shown with independent re-randomisations rB , so the shared base is not visible at spend time.

7.3 The cost of a shared base

Sharing B across a batch is what makes (23) possible, but it removes the guarantee of Section 5.1 that every signature has its own base, and PS unforgeability is only claimed for fresh bases. All notes of a batch are PS signatures on the same B , and PS signatures on a common base are closed under affine combination: for any scalars λ_i with $\sum_i \lambda_i = 1$,

$$\sum_i \lambda_i \sigma_i = \left(x_0 + \sum_{j=1}^3 x_j \sum_i \lambda_i m_{j,i} \right) B \quad (24)$$

is a valid signature on the attribute vector $\sum_i \lambda_i \vec{m}_i$. This is the forgery Coconut's footnote 3 warns about [9].

The attack end to end. A user requests two notes in one batch with amounts $v_1 \neq v_2$, both range-proven, and the same spend condition auth , so $m_{3,1} = m_{3,2}$. After aggregation the user holds (B, σ_1) and (B, σ_2) . For any target amount v^* the user picks $\lambda = (v^* - v_2)/(v_1 - v_2)$ and forms $\sigma^* = \lambda \sigma_1 + (1 - \lambda) \sigma_2$, a valid signature on amount v^* , serial secret $\lambda m_{2,1} + (1 - \lambda) m_{2,2}$ and the known spend condition. The serial is fresh, so its nullifier is not in N . The user spends σ^* as the single input of a transaction with v^* in P' , and requests outputs totalling v^* , each with an honest range proof. Every check of Section 6 passes: inputs are not range-proven, and nothing else constrains v^* . The user has created v^* units from nothing, and the two original notes remain spendable.

The attack needs only two notes with equal m_3 , which the user controls, so it cannot be prevented on the user side; it is mitigated only by preventing two signatures on one base, by making the amount of an input range-proven again at spend time, or by both.

7.4 Mitigation: per-note bases with partially amortised responses

The affine combination (24) is only possible because the rows of φ_B that involve the base are shared. The rows for P and C_m use fixed generators and can be batched by the lemma for any set of notes; the rows for C_1, C_2, C_3 are the only ones that need a common map. Giving every note its own base $B_i = H_{G_1}(C_{m,i})$ as in the single-note scheme and batching only what can be batched yields the following protocol. Write the witness of note i as $\vec{w}_i = (\vec{m}_i, r_{p,i}, r_{m,i}, \vec{r}_i)$ with $\vec{m}_i = (m_{1,i}, m_{2,i}, m_{3,i})$ and $\vec{r}_i = (r_{1,i}, r_{2,i}, r_{3,i})$.

1. The prover samples $\vec{\rho}_i \xleftarrow{\$} \mathbb{K}^8$ for every note and computes the commitments $\vec{R}_i = \varphi_{B_i}(\vec{\rho}_i)$. Let $R_P = \sum_i \vec{R}_{i,1}$ and $R_{C_m} = \sum_i \vec{R}_{i,2}$ be the sums of the first two rows.
2. Challenges: $\gamma_i = H_s(\text{tag}_{\text{iss}} \parallel \vec{C}_1 \parallel \dots \parallel \vec{C}_n \parallel R_P \parallel R_{C_m} \parallel \vec{R}_{1,3..5} \parallel \dots \parallel \vec{R}_{n,3..5} \parallel i)$.
3. Responses: per note $\vec{s}_{m,i} = \vec{\rho}_{m,i} + \gamma_i \vec{m}_i \in \mathbb{K}^3$ and $\vec{s}_{r,i} = \vec{\rho}_{r,i} + \gamma_i \vec{r}_i \in \mathbb{K}^3$; shared $s_p = \sum_i (\rho_{p,i} + \gamma_i r_{p,i})$ and $s_m = \sum_i (\rho_{m,i} + \gamma_i r_{m,i})$.
4. The proof is $(\gamma_1, \{\vec{s}_{m,i}, \vec{s}_{r,i}\}_i, s_p, s_m)$: the commitments are not sent, the verifier recomputes them from the responses and the challenges and re-derives the hash. Since all γ_i come from one hash state, sending γ_1 suffices to check the whole derivation.

The verifier recomputes B_i and checks the two batched rows and the $3n$ per-note rows:

$$\sum_i s_{m_1,i} G_p + s_p H_p = R_P + \sum_i \gamma_i P_i, \quad (25)$$

$$\sum_i (s_{m_1,i} H_1 + s_{m_2,i} H_2 + s_{m_3,i} H_3) + s_m G_1 = R_{C_m} + \sum_i \gamma_i C_{m,i}, \quad (26)$$

$$s_{m_j,i} B_i + s_{r_j,i} G_1 = \vec{R}_{i,2+j} + \gamma_i C_{j,i} \quad (i \leq n, j \leq 3). \quad (27)$$

Soundness: the per-note rows are ordinary Σ -protocol instances, so two transcripts per note extract $\vec{m}_i$ and $\vec{r}_i$; the batched rows then satisfy Lemma 1, and because the same responses $s_{m_j,i}$ appear in both, the extracted $m_{1,i}$ is the amount in P_i as well as in $C_{1,i}$. Zero knowledge is

as before. Every note is signed on its own base B_i , so (24) no longer applies and the single-note analysis of Section 8 carries over unchanged.

The proof has $6n + 3$ scalars, about $192n + 96$ bytes, against 496 bytes for the fully amortised proof and $496n$ bytes for n independent proofs. The mints perform $3n$ additional small multi-scalar multiplications. For the 100-note example of Section 9 the issuance proof grows from 0.5 KB to 19.3 KB. Whether this is acceptable, or whether a logarithmic-size compressed Σ -protocol [19] should replace it, is left open. The prototype implements the fully amortised variant of Section 7.1; until it is replaced, the batched issuance must be regarded as insecure.

8 Security Analysis

We argue each goal of Section 4 for the single-note scheme of Section 5 and for batching with the mitigation of Section 7.4. The arguments are sketches in the style of Coconut’s security appendix; a full reduction is future work.

Correctness. Shown inline: equations (10), (11), (12) and the expansion of (16).

Unforgeability. Suppose an adversary corrupting fewer than t mints outputs k valid spends with distinct nullifiers whose committed amounts exceed what it was issued. Each spend contains a ψ -proof, so by special soundness we extract (m_1, m_2, r'_p) with $K' = m_1 X_1 + m_2 X_2$ and $P' = m_1 G_p + r'_p H_p$; the pairing check then says (B', σ') is a PS signature on (m_1, m_2, m_3) under the aggregate key. Distinct nullifiers mean distinct (m_1, m_2) , hence k signatures on distinct messages. Each honest issuance produced one signature, on the attributes extracted from its φ_B -proof, and those attributes had range-proven amounts; by binding of P (DL) the amount in the signature is the amount in the range proof. If the k messages are among the issued ones, the sum of the amounts is bounded by what was issued, contradicting the assumption; otherwise some message was never signed by the honest mints, and because fewer than t key shares reveal nothing about the key (Shamir), the adversary has forged a PS signature, contradicting PS unforgeability. The reduction needs each honest signature to be on a fresh base, which (5) provides via the random oracle H_{G_1} and the binding of C_m : two requests with the same C_m open to the same attributes, so they are the same message. This is the step that fails for the fully amortised batch.

Blindness. A mint’s view of an issuance is $(\vec{C}, \vec{R}, \vec{s})$ plus the range proof. P , C_m and C_j are perfectly hiding because r_p, r_m, r_j are uniform; $(\vec{R}, \vec{s})$ is simulatable without the witness (Appendix A); and the range proof is zero knowledge. Hence the view is simulatable from the range alone. Blindness holds information-theoretically for the commitments and computationally for the proofs, against any number of colluding mints.

Unlinkability. An adversary holding all key shares sees at issuance $(B, \vec{C}, \pi)$ and at spend $(B', \sigma', V, P', K', \text{auth}, \pi')$. The spend elements are determined by (B, σ) , the attributes and fresh randomness r, r', r'_p . Under DDH in G_1 , (B, rB) is indistinguishable from (B, U) for random U , and σ' is a function of B' and public data; P' is a fresh perfectly hiding commitment; K' is independent of the issuance transcript because m_2 appeared there only inside hiding commitments; and auth likewise appeared only as m_3 inside hiding commitments. Hence the spend transcript is simulatable from the public key, the attributes’ nullifier and auth , none of which the issuance transcript determines. Note that this argument needs r'_p to be fresh; the prototype’s reuse of r_p breaks it outright (Section 10).

Double-spend detection. K' is a deterministic function of (m_1, m_2) , so two spends of one note coincide. Two different notes have independent uniform m_2 , so their nullifiers coincide with probability $1/p$. A spender cannot present a different nullifier for the same note: the pairing check fixes $X_0 + K' + m_3 X_3$ to be the message under which (B', σ') verifies, and by PS unforgeability the only such messages are the issued ones; the ψ -proof then forces K' to be the attribute term of that message rather than an arbitrary element.

Value confidentiality. P and P' are perfectly hiding. $K' = m_1 X_1 + m_2 X_2$ hides m_1 under DDH in $\mathbb{G}_2$ because $m_2 X_2$ is a uniform mask; this is a one-time mask, which suffices since a note is meant to be spent once. The balance check (18) reveals only the sum relation, and δ is uniform.

What is not covered. The role of V (Section 5.8.1); the interaction between unlinkability and the public auth, which a payee could reuse across notes and thereby link its own notes (a usage question, not a scheme one); and side channels in the prototype.

9 Performance

9.1 Sizes

Table 3 lists what is transmitted per note in each phase, using compressed encodings (48 bytes for $\mathbb{G}_1$, 96 for $\mathbb{G}_2$, 32 for a scalar). Table 4 gives the proofs as a function of the number of notes n ; the range-proof row follows the Bulletproofs aggregate size $2\lceil\log_2(\ell n)\rceil$ $\mathbb{G}_1$ elements plus a constant for ℓ -bit ranges, with $\ell = 32$ as in the original estimate.

Table 3: Bytes per note in each phase, excluding proofs.

Phase	Elements	Bytes
Issuance request	$\vec{C} = (P, C_m, C_1, C_2, C_3)$	240
Signature share (per mint)	$\tilde{\sigma}_i$	48
Stored note	$B, \sigma, m_1, m_2, r_p, \text{auth}$	224
Spend	$B', \sigma', P', K', V, \text{auth}$	368

Table 4: Proof size as a function of the number of eCash notes n . The range proofs are assumed to be for a 32-bit range.

Proof	Size	Growth
Issuance, fully amortised (§7.1)	$5\mathbb{G}_1 + 8s = 496\text{ B}$	$O(1)$
Issuance, per-note bases (§7.4)	$(6n + 3)s \approx 192n\text{ B}$	$O(n)$
Issuance, independent proofs	$n(5\mathbb{G}_1 + 8s) = 496n\text{ B}$	$O(n)$
Issuance, aggregated range proof	$(2\lceil\log_2 32n\rceil + 9)\mathbb{G}_1 + 2s$	$O(\log n)$
Spend	$4ns = 128n\text{ B}$	$O(n)$

For a transaction with 100 inputs and 100 outputs the totals are: inputs $100 \times (368 + 128) = 49.6\text{ KB}$; outputs $100 \times 240 = 24.0\text{ KB}$ plus the issuance proof (0.5 KB amortised, 19.3 KB with per-note bases) plus the aggregated range proof ($33\mathbb{G}_1 + 2s = 1.6\text{ KB}$); and 32 bytes for δ . That is 75.7 KB with the amortised proof and 94.5 KB with per-note bases, against Fedimint’s 64 KB consensus limit. Spends dominate: the two $\mathbb{G}_2$ elements K' and V account for 192 of the 496 bytes per input, which is one reason to settle the role of V (Section 5.8.1). Dropping V and

the `auth` preimage (which the transaction layer may carry once per transaction) brings an input to 368 bytes and the 100/100 transaction to 62.9KB amortised.

9.2 Timings

Table 5 reports wall-clock timings of the prototype, built in release mode with the pure-Rust `bls12_381` crate, single-threaded, on one core of an AMD Ryzen AI 9 HX 370 laptop processor; each figure is the median of seven runs. Issuance timings are for a batch of n notes under the fully amortised proof; all other rows are per note and independent of n .

Table 5: Measured timings of the prototype in milliseconds.

Operation	$n = 1$	$n = 10$	$n = 100$
Issuance: statements and proof (user)	5.1	59.0	1416
Issuance: proof verification (per mint)	3.7	22.7	1027
Blind signing of the batch (per mint)	0.9	8.4	85.3
Unblind and verify one share (user, per mint)	4.0	4.0	4.1
Aggregate 5 of 7 shares (user)	1.1	1.1	1.2
Spend: re-randomise and prove (user)	7.3	7.3	7.4
Spend: verify proof and pairing (per mint)	4.6	4.6	4.7

Two observations. First, every per-note operation is in the low milliseconds, dominated by pairings (two per share verification, two per spend verification) and by the scalar multiplications of the Σ -proofs; a mint validating the 100/100 transaction above spends about 0.5s on spends and the same again on signing shares. Second, the issuance proof scales quadratically rather than linearly in the prototype: $n = 100$ costs 240 times $n = 1$ rather than 100 times. The cause is the challenge derivation (21), which re-serialises and re-hashes all n statements for each of the n challenges. Deriving all challenges from one hash state over the statements, as the mitigation of Section 7.4 already specifies, removes the quadratic term; the group operations themselves are linear in n . The user-side issuance time also includes computing the statements and an internal self-check of the proof.

10 Implementation Notes

The prototype in this repository implements Sections 5 and 7 over BLS12-381, with the following deviations from the description that are worth knowing about.

Prototype caveats

- **Spend-time commitment randomness.** The prototype reuses the issuance randomness r_p when building P' , so $P' = P$ and a spend is trivially linkable to its issuance. The scheme requires a fresh r'_p as in (14).
- **Hash-to-group.** H_{G_1} , H_s and the generators are implemented by seeding a ChaCha RNG with SHA-256 rather than a standard `hash_to_curve` ciphersuite. This is fine for unknown-discrete-logarithm generation but not interoperable.
- **Checks not implemented.** $m_3 \neq 0$, $B' \neq 0$ and the nullifier set are not enforced by the prototype; the range proof is not integrated.
- **Only the batched path is exercised.** Single-note issuance is the $n = 1$ case of Section 7, and the tests run batches of two.

11 Open Problems

1. **Batching.** Replace the fully amortised proof with the per-note base variant of Section 7.4 or a compressed Σ -protocol, and decide whether inputs should additionally be range-proven at spend time as defence in depth.
2. **Formal security proof.** Turn the sketches of Section 8 into reductions, in particular a game-based balance definition and its reduction to PS unforgeability under q -MSDH-1 with the hashed base.
3. **The element V .** Either justify it or remove it and the r' terms of (13) (Section 5.8.1); removal saves 96 bytes and one $\mathbb{G}_2$ multiplication per spend.
4. **Transaction size.** The 100/100 example exceeds the consensus limit in all variants. Options are a larger limit, fewer notes per transaction through change-making, or a smaller spend encoding.
5. **Key rotation and nullifier pruning.** Fedimint rotates keys per epoch; the nullifier set and the aggregate key need a lifecycle.
6. **Prototype fixes** listed in Section 10, and a standard hash-to-curve ciphersuite [22].

12 Conclusion

Hiding the amount of a federated eCash note is possible with the same machinery that makes Fedimint work today, threshold signing with Shamir shares and Lagrange interpolation in the exponent, once the signature is one whose signing map is linear in both key and messages. Pointcheval–Sanders signatures issued Coconut-style provide that, Pedersen commitments and Bulletproofs make the hidden amounts balanceable and bounded, and the hidden-attribute term of the verification equation doubles as a nullifier. The resulting notes are unlinkable, confidential and detectably single-spend. The remaining cost is size: inputs carry about half a kilobyte each, and the one optimisation that makes outputs cheap, a shared signature base, is also the one that must not be used. The per-note base variant restores the security argument at a size the consensus layer can likely absorb; proving the scheme formally and shrinking the spend are the next steps.

Acknowledgements. The prototype builds on the single-issuer scheme and notes by joschisan; the amortised proof and its implementation were added in this repository.

References

- [1] D. Chaum, “Blind signatures for untraceable payments,” in *Advances in Cryptology: CRYPTO '82*, Plenum, 1983.
- [2] Fedimint contributors, “Fedimint: a federated Chaumian mint,” <https://fedimint.org>, source at <https://github.com/fedimint/fedimint>. Accessed: 2026-09-30.
- [3] D. Boneh, B. Lynn, and H. Shacham, “Short signatures from the Weil pairing,” in *ASIACRYPT 2001*, LNCS 2248, Springer, 2001.
- [4] A. Boldyreva, “Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme,” in *PKC 2003*, LNCS 2567, Springer, 2003.
- [5] J. Camenisch and A. Lysyanskaya, “Signature schemes and anonymous credentials from bilinear maps,” in *CRYPTO 2004*, LNCS 3152, Springer, 2004.
- [6] D. Pointcheval and O. Sanders, “Short randomizable signatures,” in *CT-RSA 2016*, LNCS 9610, Springer, 2016.

- [7] D. Pointcheval and O. Sanders, “Reassessing security of randomizable signatures,” in *CT-RSA 2018*, LNCS 10808, Springer, 2018.
- [8] A. Lysyanskaya, R. Rivest, A. Sahai, and S. Wolf, “Pseudonym systems,” in *SAC 1999*, LNCS 1758, Springer, 1999.
- [9] A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, “Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers,” in *NDSS 2019*, Internet Society, 2019. <https://arxiv.org/abs/1802.07344>.
- [10] Nym Technologies SA, “The Nym network,” <https://nymtech.net/nym-whitepaper.pdf>, 2021. Accessed: 2026-07-28.
- [11] A. Rial and A. M. Piotrowska, “Compact and divisible e-cash with threshold issuance,” *Proceedings on Privacy Enhancing Technologies*, 2023. <https://arxiv.org/abs/2303.08221>.
- [12] Nym Technologies SA, “nym_offline_compact_ecash,” Rust crate in the `nymtech/nym` repository (`common/nym_offline_compact_ecash`), 2024. Accessed: 2026-09-16.
- [13] T. P. Pedersen, “Non-interactive and information-theoretic secure verifiable secret sharing,” in *CRYPTO ’91*, LNCS 576, Springer, 1992.
- [14] G. Maxwell, “Confidential transactions,” 2015. <https://elementsproject.org/features/confidential-transactions/investigation>.
- [15] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, “Bulletproofs: Short proofs for confidential transactions and more,” in *IEEE Symposium on Security and Privacy*, 2018.
- [16] J. Camenisch, S. Hohenberger, and A. Lysyanskaya, “Compact e-cash,” in *EUROCRYPT 2005*, LNCS 3494, Springer, 2005.
- [17] D. Hopwood, S. Bowe, T. Hornby, and N. Wilcox, “Zcash protocol specification,” <https://zips.z.cash/protocol/protocol.pdf>.
- [18] M. Bellare, J. A. Garay, and T. Rabin, “Fast batch verification for modular exponentiation and digital signatures,” in *EUROCRYPT ’98*, LNCS 1403, Springer, 1998.
- [19] T. Attema and R. Cramer, “Compressed Σ -protocol theory and practical application to plug & play secure algorithmics,” in *CRYPTO 2020*, LNCS 12172, Springer, 2020.
- [20] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *CRYPTO ’86*, LNCS 263, Springer, 1987.
- [21] A. Shamir, “How to share a secret,” *Communications of the ACM*, 22(11), 1979.
- [22] A. Faz-Hernández, S. Scott, N. Sullivan, R. S. Wahby, and C. A. Wood, “Hashing to elliptic curves,” RFC 9380, IETF, 2023.

A Σ -protocols for Linear Maps

Both proofs of the scheme are instances of one protocol: a proof of knowledge of a preimage under a linear map. Let V and W be finite-dimensional $\mathbb{K}$ -vector spaces, in our instantiations $V = \mathbb{K}^k$ and W a product of pairing groups, and let $\varphi : V \rightarrow W$ be linear. The public statement is $Y \in \text{im } \varphi$; the prover knows $\vec{x} \in V$ with $\varphi(\vec{x}) = Y$. The full set of witnesses is the affine subspace $\vec{x} + \ker \varphi$. The witness is the prover’s private input and is never transmitted.

Interactive protocol.

commit:	$\vec{\rho} \xleftarrow{\$} V, \quad R = \varphi(\vec{\rho}),$	prover $\rightarrow$ verifier: R
challenge:	$\gamma \xleftarrow{\$} \mathbb{K},$	verifier $\rightarrow$ prover: γ
respond:	$\vec{s} = \vec{\rho} + \gamma \vec{x},$	prover $\rightarrow$ verifier: $\vec{s}$
verify:	$\varphi(\vec{s}) \stackrel{?}{=} R + \gamma Y.$	

Properties.

Completeness. $\varphi(\vec{s}) = \varphi(\vec{\rho}) + \gamma \varphi(\vec{x}) = R + \gamma Y.$

Special soundness. Two accepting transcripts $(R, \gamma, \vec{s})$ and $(R, \gamma', \vec{s}')$ with $\gamma \neq \gamma'$ give $\varphi(\vec{s} - \vec{s}') = (\gamma - \gamma')Y$, so $(\gamma - \gamma')^{-1}(\vec{s} - \vec{s}')$ is a witness. The knowledge error is $1/p$.

Honest-verifier zero knowledge. A simulator samples $\vec{s} \xleftarrow{\$} V$ and $\gamma \xleftarrow{\$} \mathbb{K}$ and sets $R = \varphi(\vec{s}) - \gamma Y$; the transcript is distributed exactly as a real one. Since $\vec{s}$ is uniform on V , nothing about which element of $\vec{x} + \ker \varphi$ the prover holds leaks; in particular $\vec{s}$ is a one-time pad of $\gamma \vec{x}$ under $\vec{\rho}$.

Fiat–Shamir. The protocol is public-coin, so the challenge can be computed as $\gamma = H_s(\text{tag} \parallel Y \parallel R)$ with H_s a random oracle [20]. The proof is $(R, \vec{s})$, or in the compact form $(\gamma, \vec{s})$ with the verifier recomputing $R = \varphi(\vec{s}) - \gamma Y$ and re-deriving γ . Hashing Y binds the proof to the statement; the domain tag prevents a proof for one map from being replayed against another. Soundness in the random-oracle model follows from special soundness by the forking lemma, with a loss linear in the number of hash queries.

Instances in this paper.

Proof	Map	Form	Domain tag
Issuance (§5.3)	$\varphi_B : \mathbb{K}^8 \rightarrow \mathbb{G}_1^5$	$(R, \vec{s})$	$\dots_ISSUANCE$
Spend (§5.8)	$\psi : \mathbb{K}^3 \rightarrow \mathbb{G}_1 \times \mathbb{G}_2$	$(\gamma, \vec{s})$	$\dots_SPEND$

Schnorr identification is the instance $\varphi(x) = xG$ and a Pedersen opening proof is $\varphi(m, r) = mG + rH$; the two proofs of the scheme differ from these only in the choice of φ .